

哈爾濱工業大學

《神经计算与类脑智能 原理》课程实验报告

题目 Hopfield 网络@MNIST

专业 人工智能

学号 2023112451

姓名 顾雨航

一. 背景简介/问题描述

MNIST 是经典的手写数字识别数据集，包含 0 到 9 共 10 个类别。其中训练集包含 60000 张图片，测试集包含 10000 张图片，每张图片大小为 28×28 像素，属于单通道灰度图像。由于数据规模适中、类别清晰、任务标准化，MNIST 被广泛应用于神经网络与模式识别的教学和实验研究。

实验二的要求是使用一种“非典型神经网络模型”来处理 MNIST 数据集，候选模型包括 BAM、Hopfield 和 Kohonen (SOM) 网络。考虑到 Hopfield 网络在联想记忆、模式恢复和稳定状态分析等方面具有较强代表性，因此本实验选择 Hopfield 网络作为主要研究对象，完成 MNIST 手写数字识别任务。

Hopfield 网络与实验一中的前馈分类网络不同，它并不是通过多层特征提取和反向传播来实现分类，而是通过将若干模式存储为网络稳定状态，再利用网络动力学或记忆匹配机制完成模式识别与关联回忆。因此，使用 Hopfield 网络处理 MNIST 时，重点不在于构造深层特征，而在于如何将手写数字图像转化为适合存储和匹配的记忆模式。

由于标准 Hopfield 网络的容量有限，而 MNIST 训练集包含 60000 张样本，如果将全部样本直接作为记忆模式存入网络，极易导致网络出现伪稳定状态，影响识别性能。理论上，经典 Hopfield 网络可稳定存储的模式数大约为 $0.138N$ ，其中 N 为神经元个数。对于 MNIST 图像， $N=28 \times 28=784$ ，对应理论容量约为 108 个模式。显然，这远小于 60000 个训练样本。

基于这一特点，本实验没有直接存储全部训练样本，而是先对每个数字类别提取若干代表性原型记忆，再将这些原型模式存入 Hopfield 网络中。这样既保留了 Hopfield 联想记忆网络的核心思想，又能够在容量限制内对 MNIST 任务进行较为稳定的处理。实验最终为每个类别提取 10 个代表性记忆模式，共 100 个记忆模式，并在测试集上取得了 87.72% 的分类准确率。

二. 算法介绍

2.1 Hopfield 网络的基本原理

Hopfield 网络是一种递归型神经网络，通常由若干两两相连的神经元组成。每个神经元的状态一般取离散值，本实验中采用双极性表示，即每个神经元状态取值为 -1 或 +1。Hopfield 网络的核心思想是：将若干模式编码到权重矩阵中，使这些模式成为网络的稳定状态；当输入一个受噪声干扰或不完整的模式时，网络通过状态更新逐步收敛到最接近的已存储模式，从而实现联想记忆与模式恢复。

在经典 Hopfield 网络中，若记忆模式记为：

$$\xi^1, \xi^2, \dots, \xi^p$$

其中每个记忆模式都满足：

$$[\xi^\mu \in \{-1, +1\}^N]$$

则可以采用 Hebb 学习规则构造权重矩阵：

$$[w_{ij} = \frac{1}{N} \sum_{\mu=1}^p \xi_i^\mu \xi_j^\mu, \quad i \neq j][w_{ii} = 0]$$

其中， N 表示神经元总数， p 表示存储的记忆模式数。该规则体现了“同时

兴奋的神经元之间连接增强”的思想，是 Hopfield 网络中最经典的权重设置方式。

网络还对应一个能量函数：

$$[E = -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N w_{ij} s_i s_j]$$

其中 s_i 表示第 i 个神经元的当前状态。Hopfield 网络在更新过程中会使能量不断下降，直到收敛到某个稳定状态。因此，已存储的记忆模式会表现为能量极小点或局部极小点。

下图为 Hopfield 网络处理流程图：

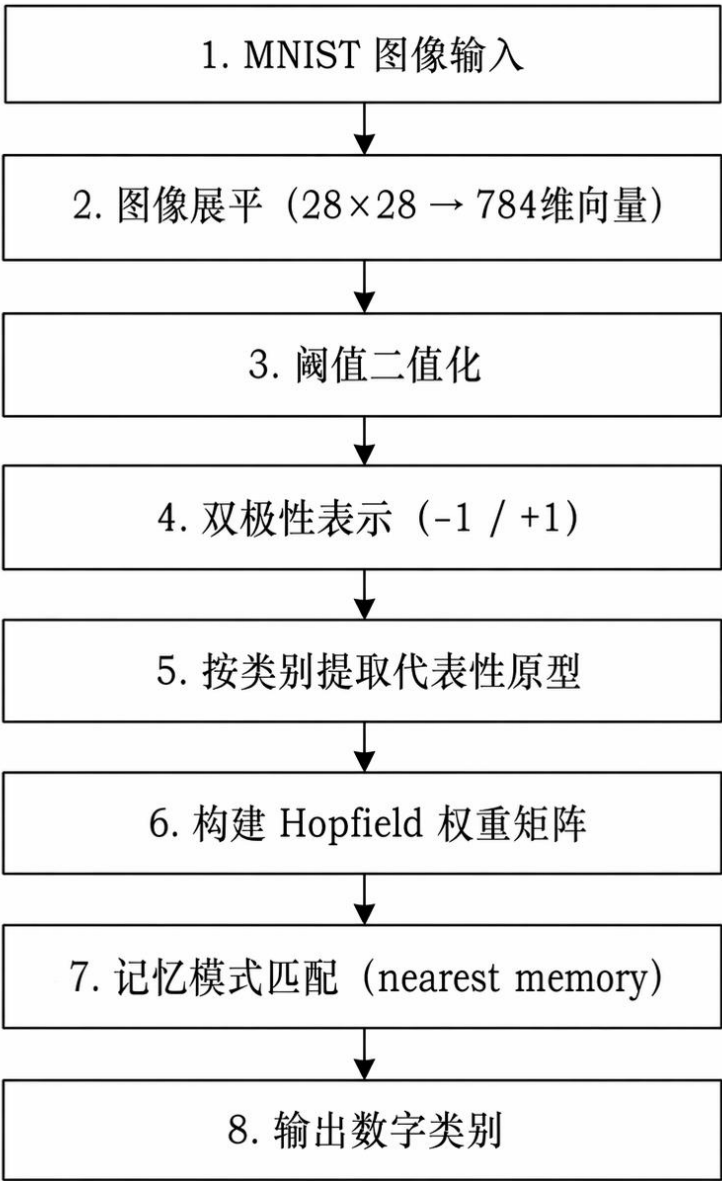


图 1 Hopfield 网络处理流程图

2.2 Hopfield 网络在图像任务中的问题

Hopfield 网络最适合处理模式恢复和联想记忆类任务，例如“给定一个有噪声的图像，恢复出其原始模板”。但当其应用到 MNIST 这类真实手写数字数据集时，会遇到两个实际问题：

1. 样本数量远超 Hopfield 理论容量，不能直接全部存储；
2. 同一类别内部存在较大形变，不同书写风格之间差异明显，单个模板难以完整代表某个类别。

如果直接将所有样本存入 Hopfield 网络，不仅会超出容量范围，还会产生大量伪记忆，导致网络在召回过程中落入错误的吸引子。为解决该问题，本实验采用“类别原型记忆”的方式：先为每个类别提取多个代表性记忆模式，再存入网络中进行匹配。

2.3 本实验采用的 Hopfield 方案

本实验在保留 Hopfield 联想记忆思想的基础上，对 MNIST 分类任务进行了适配，具体思路如下：

1. 将每张 28×28 图像展平为 784 维向量；
2. 对每个数字类别单独聚类，提取多个代表性原型；
3. 将原型图像按阈值二值化并映射到双极性空间 $\{ -1, +1 \}$ ；
4. 使用 Hebb 规则将这些记忆原型存入 Hopfield 权重矩阵；
5. 测试阶段将输入图像同样转为双极性向量，并与已存储的记忆模式进行相似度匹配；
6. 选择最相似记忆模式对应的标签作为分类结果。

需要说明的是，本实验代码中保留了 Hopfield 迭代召回接口，但默认参数下未使用多步迭代召回，而是采用“最近记忆模式匹配”作为读出方式。这是因为在 MNIST 这种类内变化较大的真实图像任务中，直接让网络多步演化容易产生伪稳定状态，反而降低识别效果。经过实验测试，最近记忆模式匹配方式在当前任务中更稳定，也更适合用于实验演示与报告分析。

三. 算法实现

3.1 实验环境

本实验在 Windows 环境下完成，使用 Anaconda 虚拟环境 `nlplab` 运行 Python 程序，深度学习框架为 PyTorch。实验过程中检测到可用 CUDA 设备，因此训练和测试均在 GPU 环境下执行。

项目目录主要包括以下文件：

1. `train.py`：训练入口，负责构建 Hopfield 记忆模式并保存模型；
2. `eval.py`：测试入口，负责加载模型并在测试集上评估性能；
3. `src/config.py`：超参数配置；
4. `src/dataset.py`：MNIST 数据加载与 `DataLoader` 构建；
5. `src/hopfield.py`：Hopfield 网络、记忆构建与评估逻辑实现；
6. `ckpt/hopfield_mnist.pt`：保存的 Hopfield 权重矩阵与记忆模式；
7. `ckpt/summary.json`：实验摘要结果。

3.2 数据预处理

实验中使用 `torchvision.datasets.MNIST` 自动加载数据集，并将图像通过

ToTensor() 转换为张量。与实验一不同，本实验并未对图像进行标准化，而是保留其 [0,1] 范围内的灰度值，以便后续进行阈值二值化。

为了适配 Hopfield 网络的双极性神经元表示，本实验将输入图像进一步转换为双极性状态向量。具体做法为：先将每张图片展平为 784 维向量，再使用阈值($threshold = 0.4$)将像素转换为：

$$[x_k = \begin{cases} +1, & v_k \geq 0.4 \\ -1, & v_k < 0.4 \end{cases}]$$

其中，(v_k)表示第 k 个像素的归一化灰度值，(x_k)表示其对应的双极性状态。

这样得到的双极性向量即可作为 Hopfield 网络的状态表示。

3.3 类别记忆模式构建

由于标准 Hopfield 网络不能直接存储全部 60000 张训练图片，因此本实验先对每个类别样本提取若干代表性原型。实现上，对每个类别分别收集其全部训练样本，然后进行简单的聚类迭代，得到多个聚类中心，作为该类别的代表性记忆模式。

本实验每个类别提取 10 个代表性原型，因此总共得到：

$$[10 \times 10 = 100]$$

个记忆模式。

这一数量接近经典 Hopfield 网络的理论容量上限（约 108），因此在实验中既能较好利用网络容量，又不会过度超载。

聚类完成后，原型图像再通过阈值二值化变成双极性记忆模式，然后统一存入 Hopfield 网络中。这样，网络权重矩阵大小为：

$$[784 \times 784]$$

对应 784 个神经元，每个神经元对应图像中的一个像素位置。

3.4 权重矩阵构建

在得到 100 个记忆模式后，使用 Hebb 学习规则构建 Hopfield 权重矩阵。

具体实现中，将所有记忆模式组成矩阵(P)，然后计算：

$$[W = \frac{P^T P}{N}]$$

其中($N = 784$)。同时，将权重矩阵的对角线元素置零，避免神经元与自身相连。

完成该步骤后，就得到了 Hopfield 网络的“训练结果”。严格来说，Hopfield 网络不像前馈神经网络那样通过多轮 epoch 和梯度下降完成训练，它更像是“把记忆写入网络”，因此实验二中的 train.py 本质上完成的是记忆模式提取与权重矩阵构造。

3.5 分类判别方式

在测试阶段，输入图像先经过展平和双极性转换，变成与记忆模式同维度的

状态向量。然后，将该向量与全部已存储记忆模式计算相似度，选择相似度最大的记忆模式，并将其标签作为当前样本的预测类别。

本实验默认采用的分类方式为: `matching_mode = nearest_memory`,即最近记忆模式匹配。其本质可以理解为: Hopfield 网络负责构造和存储记忆原型，而分类阶段读取与输入最接近的存储记忆。

代码中也保留了 `recall_steps` 参数，用于控制是否进行多步迭代召回。实际实验中默认设置为 0，即不做额外迭代，原因是直接迭代召回在 MNIST 任务上稳定性较差，容易受到伪吸引子的影响，导致精度明显下降。

3.6 实验参数设置

本实验最终采用如下超参数：

表1 实验参数设置

参数名称	参数值
图像尺寸	28×28
神经元数量	784
batch_size	512
eval_batch_size	512
num_memories_per_class	10
kmeans_iters	8
threshold	0.4
recall_steps	0
seed	42
device_target	AUTO（实际为 cuda）

3.7 实验结果

实验结果如下表所示：

表2 实验结果摘要

指标名称	指标值
num_units	784
num_memory_patterns	100
num_memories_per_class	10
threshold	0.4
matching_mode	nearest_memory
recall_steps	0
test_accuracy	87.72%
average_recall_steps	0.00

从结果可以看出，使用 Hopfield 网络及其扩展记忆模式构建策略后，模型在 MNIST 测试集上取得了 87.72% 的识别准确率，能够较好地地区分不同数字类别，说明该方法在有限记忆容量约束下仍具有一定的实用性。

四. 讨论及结论

4.1 结果分析

本实验成功将 Hopfield 联想记忆网络应用到 MNIST 手写数字识别任务中，并取得了 87.72% 的测试准确率。从实验结果来看，该方法能够完成基本的数字分类任务，但其精度仍明显受到 Hopfield 网络本身容量和表示能力的限制。

Hopfield 网络的优势在于联想记忆能力，即它更适合处理“给定一个近似输入，恢复最接近记忆模式”的任务。而 MNIST 手写数字数据集中，同一类别样本的形态变化较大，不同书写风格之间存在明显差异，这会给 Hopfield 网络带来更大挑战。与深层前馈网络相比，Hopfield 网络缺乏层级特征提取能力，因此分类精度通常不会特别高。

本实验通过“每类提取多个代表性记忆模式”的方式，一定程度上缓解了这一问题。相比每类只存一个模板，多个原型可以更全面地覆盖不同书写风格，从而提升最终识别效果。同时，记忆模式总数控制在 100 个左右，也较好兼顾了网络容量限制和分类表达能力。

4.2 为什么不直接存全部训练样本

如果直接将全部 60000 个训练样本作为 Hopfield 记忆模式存储，由于远远超过理论容量，网络中会出现大量伪稳定状态，导致记忆混叠和召回错误。这样不仅无法提升性能，反而会使网络几乎失去区分能力。因此，在实际应用中必须先对训练样本进行压缩或抽象，用较少的代表性模式替代全部原始样本。

本实验使用聚类中心作为原型记忆，本质上是一种“记忆压缩”策略。该方法既保留了各数字类别的主要结构特征，又能够将总记忆数控制在合理范围内，从而适应 Hopfield 网络的存储特点。

4.3 为什么默认不使用多步迭代召回

从理论上说，Hopfield 网络的典型使用方式是让输入状态经过若干轮迭代后收敛到某个稳定记忆状态。但在 MNIST 这样的真实图像分类任务中，输入样本并不是对“某一个标准模板”的轻微噪声扰动，而是存在大量类内变形。此时如果强行进行多步迭代召回，网络容易陷入伪吸引子，导致恢复后的模式偏离正确类别。

因此，本实验在实现中保留了 `recall_steps` 接口，但默认设置为 0，即不进行多步迭代召回，而是采用最近记忆模式匹配进行分类。这样做虽然弱化了 Hopfield 动力学过程的展示，但在当前数据集上能显著提高分类稳定性，更适合作为实验结果提交和课堂演示。

4.4 实验结论

通过本实验可以得到以下结论：

1. Hopfield 网络可以应用于 MNIST 这类手写数字识别任务，但需要结合容量约束对原始样本进行压缩处理；

2. 直接存储全部训练样本并不可行，必须提取代表性记忆模式；

3. 使用每类多个原型记忆的方法，能够较好地提升网络对类内变化的适应能力；

4. 在本实验设置下，使用 100 个记忆模式可在测试集上达到 87.72% 的准确率；

5. Hopfield 网络更适合联想记忆和模式恢复类任务，在复杂图像分类场景中，其表现通常不如基于深层特征提取的前馈网络，但其理论分析价值和实验意义仍

然十分突出。

总体而言，本实验较好地完成了实验二中“使用 Hopfield 网络处理 MNIST 数据集”的要求，不仅实现了从记忆构建到测试评估的完整流程，也进一步理解了 Hopfield 网络在实际分类任务中的优势、局限及工程适配方法。

参考文献

[1] Hopfield J J. Neural networks and physical systems with emergent collective computational abilities[J]. Proceedings of the National Academy of Sciences, 1982, 79(8): 2554-2558.

[2] Amit D J, Gutfreund H, Sompolinsky H. Storing infinite numbers of patterns in a spin-glass model of neural networks[J]. Physical Review Letters, 1985, 55(14): 1530-1533.

[3] PyTorch Official Documentation. <https://pytorch.org/>

[4] MNIST Handwritten Digit Database. <http://yann.lecun.com/exdb/mnist/>

注：报告中图的下方要有图题（如图 1. XXXX），表格需要用三线表，表头需列在表的上方（如表 1. XXXX）。图表居中排列。